# WEB-BASED TOOL FOR DETECTING AND REDACTING PII(PERSONAL IDENTIFIABLE INFORMATION) FROM PDF FILES

A Mini Project Report

Submitted in partial fulfillment of the requirements for the award of the degree of

**Bachelor of Engineering**

in

**Information Technology**

By


| | |
|---|---|
| **ALLADI SRIJA** | **2456-22-737-067** |
| **NALUVALA SUCHITHA** | **2456-22-737-105** |
| **PALAGORLA MANASA** | **2456-22-737-305** |

**Under the guidance of**

**K. Ganga Swathi**

**Assistant Professor**

**DEPARTMENT OF INFORMATION TECHNOLOGY**

**Gokaraju Lailavathi Engineering College**

**(Affiliated to Osmania University, Approved by AICTE)**

**Nizampet, Hyderabad-500090 (2024-2025)**

## Department of Information Technology

## Gokaraju Lailavathi Engineering College

(Affiliated to Osmania University, Approved by AICTE)

Nizampet, Hyderabad-500090


(2024-2025)

# CERTIFICATE

This is to Certify that A Mini Project report entitled **"Web-Based Tool for Detecting and Redacting PII(Personal Identifiable Information) from PDF Files**" is being submitted by Alladi Srija **(**2456-22-737-067**),** Naluvala Suchitha **(**2456-22-737-105**),** Palagorla Manasa (2456-22-737-305) in partial fulfillment of the requirement of the award for the degree of Bachelor of Engineering in "Information Technology" O.U., Hyderabad during the year 2024-2025 is a record of bonafide work carried out by them under my guidance. The results presented in this project have been verified and are found to be satisfactory.

| Project Guide | H.O.D |
|---|---|
| **K. Ganga Swathi** | **Dr. Padmalaya Nayak** |
| Assistant Professor, | Professor & Head, |
| Dept of CSE | Dept. of CSE |

**External Examiner(s)**

## Department of Information Technology

# Gokaraju Lailavathi Engineering College

(Affiliated to Osmania University, Approved by AICTE)

Nizampet, Hyderabad-500090


(2024-2025)

## DECLARATION

We, Alladi Srija bearing Ht.No. 2456-22-737-067, Naluvala Suchitha bearing Ht.No. 2456-22-737-105 and Palagorla Manasa bearing Ht.No. 2456-22-737-305 hereby certify that the mini project entitled "**WEB-BASED TOOL FOR DETECTING AND REDACTING PII(PERSONAL IDENTIFIABLE INFORMATION) FROM PDF FILES**" is submitted in the partial fulfilment of the required for the award of the degree of Bachelor of Engineering in Information Technology.

This is a record work carried out by us under the guidance of K.Ganga Swathi, Assistant Professor, CSE, Gokaraju Lailavathi Engineering College, Nizampet.The results embodied in this report have not been reproduced/copied from any source. The results embodied in this report have not been submitted to any other university or institute for the award of any other degree or diploma.

| | |
|---|---|
| **Alladi Srija** | **2456-22-737-067** |
| **Naluvala Suchitha** | **2456-22-737-105** |
| **Palagorla Manasa** | **2456-22-737-305** |

# ACKNOWLEDGEMENT

It is our privilege and pleasure to express my profound sense of respect, gratitude and indebtedness to our guide Ms.K.Ganga Swathi, Assistant Professor, Department of Computer Science and Engineering, Gokaraju Lailavathi Engineering College, for her inspiration, guidance, cogent discussion, constructive criticisms and encouragement throughout this dissertation work.

We express our sincere thanks to Project Coordinator Ms.S.Priyanka, Assistant Professor, Department of Computer Science and Engineering, Gokaraju Lailavathi Engineering College, for her valuable suggestions and constant helping in completing the work.

We express our sincere thanks to Dr. A. Sai Hanuman, Principal, Gokaraju Lailavathi Engineering College, Nizampet, Hyderabad, for his encouragement and constant help.

We extend our sincere thanks to all the teaching and non-teaching staff of Information Technology Department for their support and encouragement.

Last but not least, we wish to acknowledge my friends and family members for giving moral strength moral strength and helping us to complete this dissertation.

# ABSTRACT

In today's digital landscape, the protection of sensitive information is a growing concern, especially with the increasing exchange of documents containing Personally Identifiable Information (PII). This project proposes a robust system for the automated detection and redaction of PII from PDF documents using a hybrid approach involving regular expressions, contextual validation, and PDF processing techniques. The system is designed to identify critical data types such as Aadhaar numbers, PAN numbers, phone numbers, and email addresses, ensuring accuracy through additional validation, particularly for Aadhaar patterns.

Built using Flask for the web interface and PyMuPDF (Fitz) for precise PDF manipulation, the system allows users to upload a document, view detected PII, choose which elements to redact, and download a clean, redacted version. Unlike conventional detection-only systems or manual redaction tools, this project offers end-to-end privacy preservation, combining accuracy, user control, and security. This solution is especially useful in contexts where sensitive data must be shared without compromising confidentiality, such as legal, financial, and governmental documentation.

# CONTENTS

**Chapter-1 INTRODUCTION .....1**

1.1 MOTIVATION… ..... 1

1.2 PROBLEM STATEMENT..... 1

1.3 PROJECT OBJECTIVE .....1

**Chapter-2 LITERATURE SURVEY ..... 3**

2.1 PII DETECTION USING NLP AND NER-3 .....3

2.2 MACHINE LEARNING AND DEEP LEARNING APPROACHES..... 3

2.3 LIMITATIONS IN EXISTING REDACTION SYSYTEMS.....4

2.4 CHALLENGES IN REGEX-BASED DETECTION .....4

2.5 CURRENT RESERCH GAPS .....4

2.6 PROPOSED SYSTEM ADAVANTAGES……..4

2.7 CONCLUSION.....4

**Chapter-3 SOFTWARE REQUIREMENT SPECIFICATION ..... 5**

3.1 SOFTWARE REQUIREMENTS .....5

3.2 HARDWARE REQUIREMENTS.....5

3.3 FUNCTIONAL REQUIREMENTS .....5

3.4 NON-FUNCTIONAL REQUIREMENTS .....6

**Chapter-4 SYSTEM DESIGN..... 7**

4.1 SYSTEM DESIGN.....7

4.2 PROPOSED METHODOLOGY .....8

4.3 UML DESIGN.....9

4.3.1 CLASS DIAGRAM….....11
4.3.2 USECASE DIAGRAM….....12

4.3.3 COMPONENT DIAGRAM….....13

4.3.4 SEQUENCE DIAGRAM… ..... 14

4.3.5 ACTIVITY DIAGRAM.....15

4.4 TECHNOLOGY DESCRIPTION .....16

**Chapter-5 IMPLEMENTATION ..... 17**

5.1 DEVELOPMENT ENVIRONMENT SETUP…….17

5.2 PDF UPLOAD AND PREPROCESSING……17

5.3 AUTOMATED PII DETECTION……17

5.4 USER-CONTROLLED REDACTION……18

5.5 REDACTION PROCESS…….18

5.6 SECURITY AND FILE MANAGEMENT……..19

5.7 INTEGRATION AND TESTING……19

5.8 DEPLOYMENT……19

**Chater-6 TESTING……..31**

6.1 UNIT TESTING……31

6.2 INTEGRATION TESTING……...33

**Chapter-7 RESULTS……..35**

**Chapter-8 CONCLUSION ..... 40**

8.1 CONCLUSION… ..... 40

8.2 FUTURE SCOPE… .....40

**REFERENCES…… 42**

**PLAGIARISM TEST……...43**

| SNO. | LIST OF FIGURES | PAGE NO. |
|---|---|---|
| 4.2.1 | Proposed Methodology | 8 |
| 4.3.1 | UML Hierarchy Diagrams | 10 |
| 4.3.1.1 | Class Diagram | 11 |
| 4.3.2.1 | Use Case Diagram | 12 |
| 4.3.3.1 | Component Diagram | 13 |
| 4.3.4.1 | Sequence Diagram | 14 |
| 4.3.5.1 | Activity Diagram | 15 |
| 7.1 | Home Page Interface | 38 |
| 7.2 | PDF Upload Section | 38 |
| 7.3 | Detected Information Section | 38 |
| 7.4 | Download Redacted PDF | 38 |

# CHAPTER-1

# INTRODUCTION

## 1.1 MOTIVATION

In response to the growing risk of exposing sensitive personal data in digital documents, which can lead to privacy breaches and identity theft, this project aims to implement a reliable and automated redaction solution. By combining regular expressions with contextual validation and PDF manipulation using PyMuPDF, the system provides accurate detection and permanent redaction of Personally Identifiable Information (PII) such as Aadhaar numbers, PAN details, phone numbers, and email addresses. Through a user-friendly Flask-based web interface, the project enhances document security, reduces human error, and ensures the safe handling of confidential information in sensitive domains like legal, financial, and governmental sectors.

## 1.2 PROBLEM STATEMENT

In today's digital landscape, documents containing sensitive personal information such as Aadhaar numbers, PAN numbers, phone numbers, and email addresses are frequently shared across sectors like government, finance, healthcare, and education. The accidental exposure of such data can lead to identity theft, fraud, and serious privacy violations. Existing redaction methods face several limitations: manual redaction is time-consuming and error-prone, while many automated tools rely on cloud infrastructure, raising security concerns and risking data leaks.Additionally, automated systems often lack contextual understanding, leading to false positives or missed PII, and typically do not allow selective redaction—forcing users to remove all detected data without discretion. Many tools also fail to validate data against standard formats, increasing the chance of incorrect redaction. Therefore, there is a need for a secure, offline, and user-friendly redaction system that combines pattern recognition with validation, ensures accurate and customizable redaction, and maintains the integrity of the original document for real-world applications.

## 1.3 PROJECT OBJECTIVE

The objective of this project is to develop and deploy an automated system for detecting and redacting Personally Identifiable Information (PII) from PDF documents using regular expressions, contextual validation, and PDF manipulation via PyMuPDF. The system aims to accurately identify

sensitive data types such as Aadhaar numbers, PAN numbers, phone numbers, and email addresses, ensuring compliance with data privacy standards.Additionally, the project focuses on providing a secure, user-friendly web interface built with Flask that allows users to upload documents, review detected PII, and selectively redact sensitive information based on individual needs. The redaction process ensures permanent removal of data, preserving both document structure and confidentiality. By achieving these objectives, the project intends to enhance document privacy, reduce manual effort, and support secure information sharing in sectors like legal, financial, governmental, and educational domains.

# CHAPTER 2
# LITERATURE SURVEY

The growing concern around data privacy in the digital age has intensified the need to protect Personally Identifiable Information (PII) in documents shared across various sectors, such as healthcare, finance, government, and education. Traditional redaction methods, such as manual blackouts in tools like Adobe Acrobat, are not only time-consuming but also prone to human error and inefficiency—especially when dealing with large document volumes. This has led to increasing interest in automated techniques for PII detection and redaction.

## 2.1 PII DETECTION USING NLP AND NER

Natural Language Processing (NLP) and Named Entity Recognition (NER) have emerged as popular approaches for detecting PII in unstructured text. Tools like Microsoft Presidio, Google Cloud DLP, and Amazon Comprehend utilize pretrained models to identify common entities such as names, email addresses, IDs, and phone numbers. While these tools offer high accuracy in entity detection, they primarily operate on cloud infrastructure, raising concerns about data security, latency, and ongoing operational costs.

## 2.2 MACHINE LEARNING AND DEEP LEARNING APPROACHES

Recent studies have applied machine learning and deep learning models such as spaCy, BERT, and BiLSTM-CRF for enhanced PII detection. These models excel in understanding contextual relationships within text and are trained using annotated datasets to improve detection accuracy. Although effective, they require substantial computational resources, access to high-quality labeled data, and careful hyperparameter tuning—making them less feasible for lightweight, real-time applications.

## 2.3 LIMITATIONS IN EXISTING REDACTION SYSTEMS

Despite advancements in detection, many systems lack integrated redaction capabilities—especially for structured formats like PDF. Detection tools often require third-party software to perform

redaction, complicating the workflow. Moreover, some tools enforce blanket redaction policies, removing all detected PII without user input. This can lead to over-redaction, where essential non-sensitive information is lost, affecting document usability.

## 2.4 CHALLENGES IN REGEX-BASED DETECTION

Regex-based models, while lightweight and fast, often suffer from false positives, particularly when context is not considered. For instance, random digit sequences may be misidentified as Aadhaar numbers. Without proper validation logic, such systems struggle to distinguish between actual PII and similar-looking patterns, leading to inaccurate redactions.

## 2.5 CURRENT RESEARCH GAPS

Current solutions mostly focus on PII detection rather than offering complete end-to-end redaction workflows. Few systems validate detected entities or provide a user interface for selective redaction. Additionally, limited support for offline execution and integration with file formats like PDF further reduces their applicability in sensitive environments.

## 2.6 PROPOSED SYSTEM ADVANTAGE

To address these gaps, our project introduces a comprehensive PII detection and redaction system that operates entirely offline. It combines regex-based pattern matching with Aadhaar-specific contextual validation to reduce false positives. The system utilizes Flask for a user-friendly web interface and PyMuPDF for precise PDF manipulation, allowing users to selectively redact information. This end-to-end approach improves efficiency, enhances control, and ensures strong privacy compliance—making it suitable for real-world, privacy-sensitive applications.

## 2.7 CONCLUSION

The literature highlights the need for integrated, accurate, and user-controllable redaction systems for protecting PII in digital documents. While many tools excel in detection, they lack complete workflows for secure and flexible redaction. This project aims to bridge that gap by delivering a lightweight, offline-capable system that detects, validates, and redacts PII from PDFs, supporting safe document sharing in sensitive sectors.

# CHAPTER-3

## SOFTWARE REQUIREMENT SPECIFICATION

This chapter gives an overview of the software and hardware components required for our project.

### 3.1 SOFTWARE REQUIREMENTS

Operating System : Windows 11

Coding Language : Python 3.10.7, Html, CSS

Libraries/Frameworks : Flask, PyMuPDF (fitz), re (Regular Expressions)

Web Server : Flask development server

### 3.2 HARDWARE REQUIREMENTS

System : intel i5 or above

Storage : Sufficient storage

### 3.3 FUNCTIONAL REQUIREMENTS

These are the requirements that the end user specifically demands as basic facilities that the system should offer. All these functionalities need to be necessarily incorporated into the system as a part of the contract. These are represented or stated in the form of input to be given to the system, the operation performed, and the output expected. They are basically the requirements stated by the user which one can see directly in the final product, unlike the non-functional requirements.

- PDF File Upload
- PII Detection Using Regex and Validation
- Preview of Detected PII
- Redaction Process
- Download Redacted Document
- Error and Invalid File Handling
- Offline Functionality

## 3.4 NON-FUNCTIONAL REQUIREMENTS

These are basically the quality constraints that the system must satisfy according to the project contract. The priority or extent to which these factors are implemented varies from one project to other. They are also called non-behavioral requirements.

They basically deal with issues like:

- Portability
- Security
- Maintainability
- Reliability
- Scalability
- Performance
- Reusability
- Flexibility

# CHAPTER-4
# SYSTEM DESIGN

## 4.1 System Design

In this phase, the system and software design documents are prepared as per the requirement specification document. This helps define overall system architecture.

There are two kinds of design documents developed in this phase:

### High-Level Design (HLD)

- Brief description and name of each module
- An outline about the functionality of every module
- Interface relationship and dependencies between modules
- Database tables identified along with their key elements
- Complete architecture diagrams along with technology details

### Low-Level Design(LLD)

- Functional logic of the modules
- Database tables, which include type and size
- Complete detail of the interface
- Addresses all types of dependency issues
- Listing of error messages

## 4.2 PROPOSED METHODOLOGY:

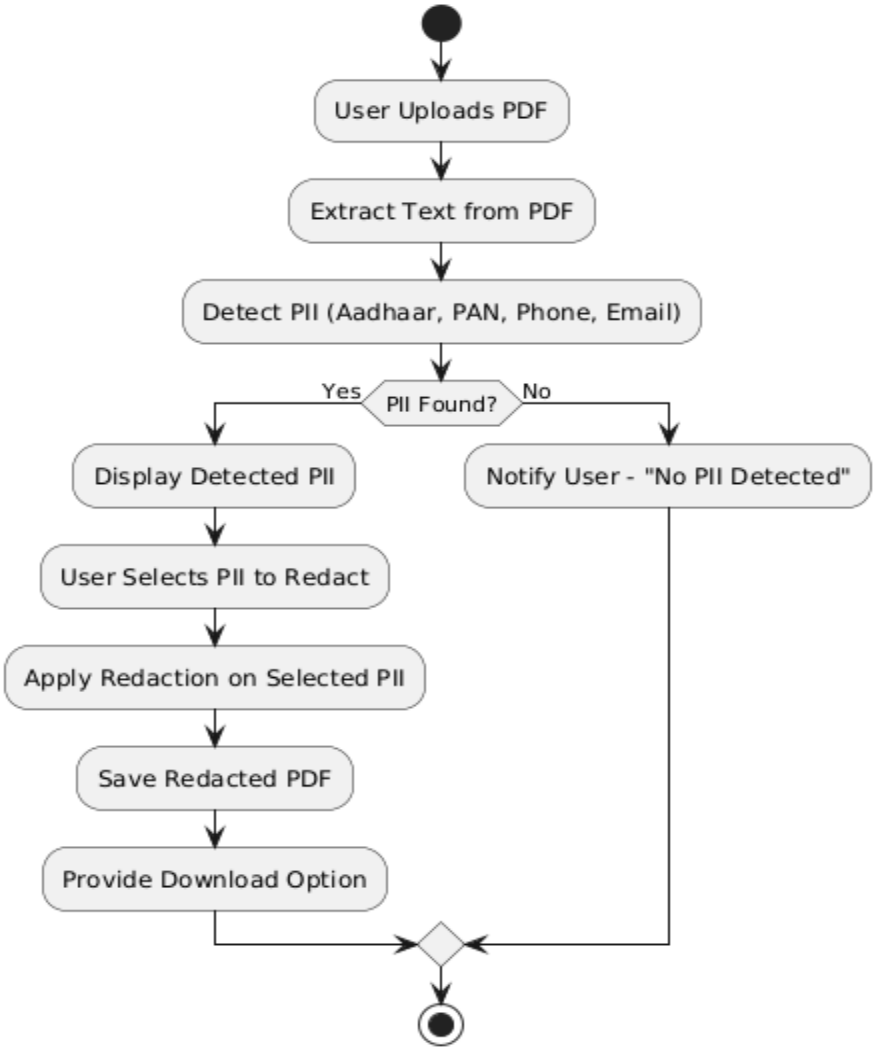


Figure-4.2.1 Proposed Methodology

## 4.3 UML Design:

Unified Modeling Language (UML) is a general purpose modeling language. The main aim of UML is to define a standard way to visualize the way a system has been designed.

UML is not a programming language; it is rather a visual language. We use UML diagrams to portray the behavior and structure of a system, UML helps software engineers, businessmen and system architects with modeling, design and analysis. The Object Management Group (OMG) adopted Unified Modeling Language as a standard in 1997. It's been managed by OMG ever since. International Organization for Standardization (ISO) published UML as an approved standard in 2005. UML has been revised over the years and is reviewed periodically.

## Do we really need UML?

- Complex applications need collaboration and planning from multiple teams and hence require a clear and concise way to communicate amongst them.
- Businessmen do not understand code. So UML becomes essential to communicate with non programmer's essential requirements, functionalities and processes of the system.
- UML is linked with object oriented design and analysis. UML makes the use of elements and forms associations between them to form diagrams. Diagrams in UML can be broadly classified as:

## The Primary goals in the design of the UML are as follows:

- Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
- Provide extendibility and specialization mechanisms to extend the core concepts.
- Be independent of particular programming languages and development process.
- Provide a formal basis for understanding the modeling language.
- Encourage the growth of OO tools market.
- Support higher level development concepts such as collaborations, frameworks, patterns and components.
- Integrate best practices.

## Types of UML Diagrams:

## Structural Diagrams:

Capture static aspects or structure of a system. Structural Diagrams include: Component Diagrams, Object Diagrams, Class Diagrams and Deployment Diagrams.

## Behavior Diagrams:

Capture dynamic aspects or behavior of the system. Behavior diagrams include: Use Case Diagrams, State Diagrams, Activity Diagrams and Interaction Diagrams.

**The image below shows the hierarchy of diagrams according to UML**

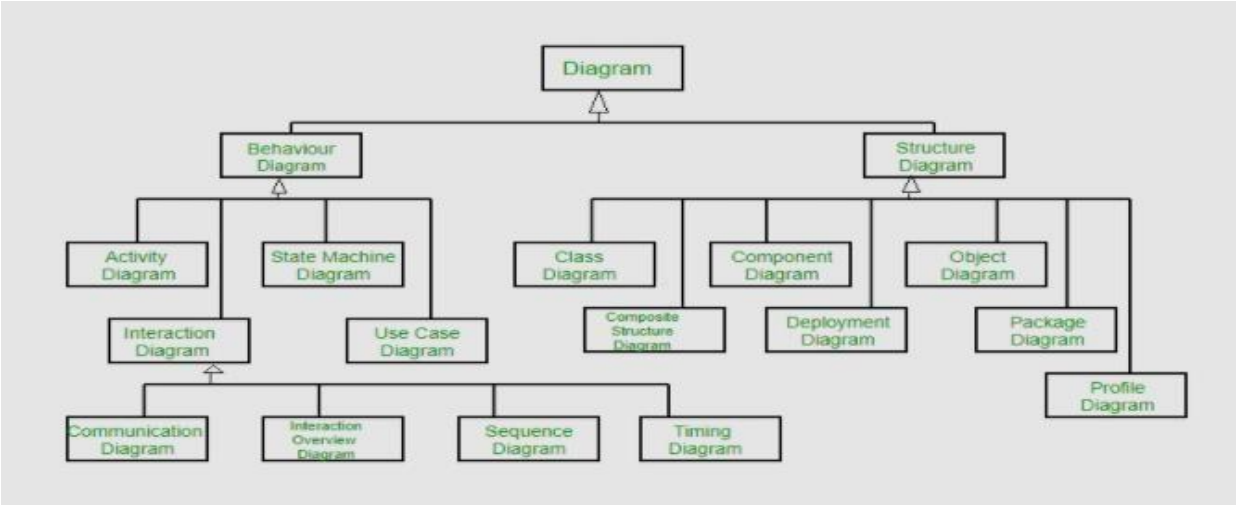


Figure-4.3.1 UML Hierarchy diagrams

### 4.3.1 CLASS DIAGRAM:

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

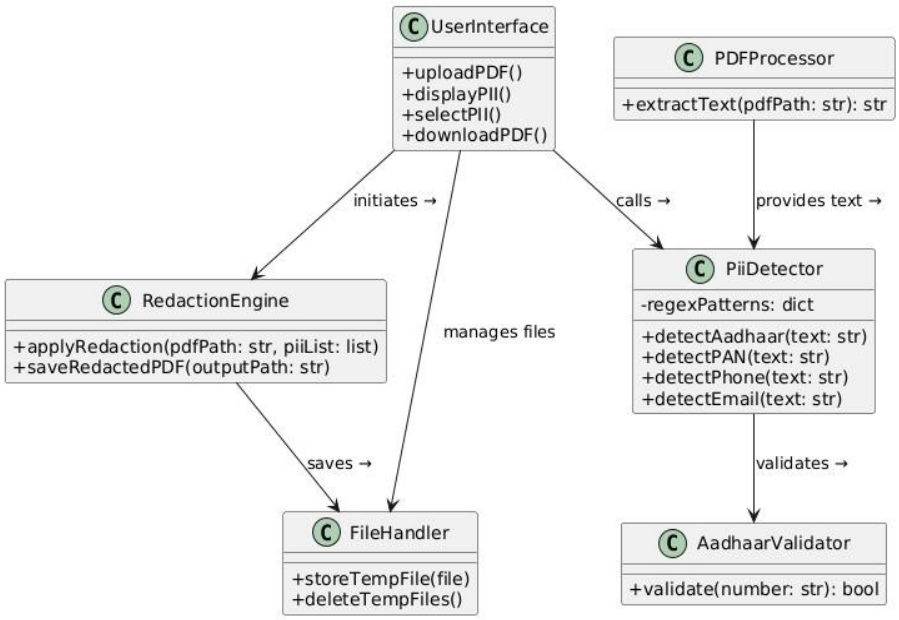


Figure-4.3.1.1 Class Diagram

### 4.3.2 USE CASE DIAGRAM

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

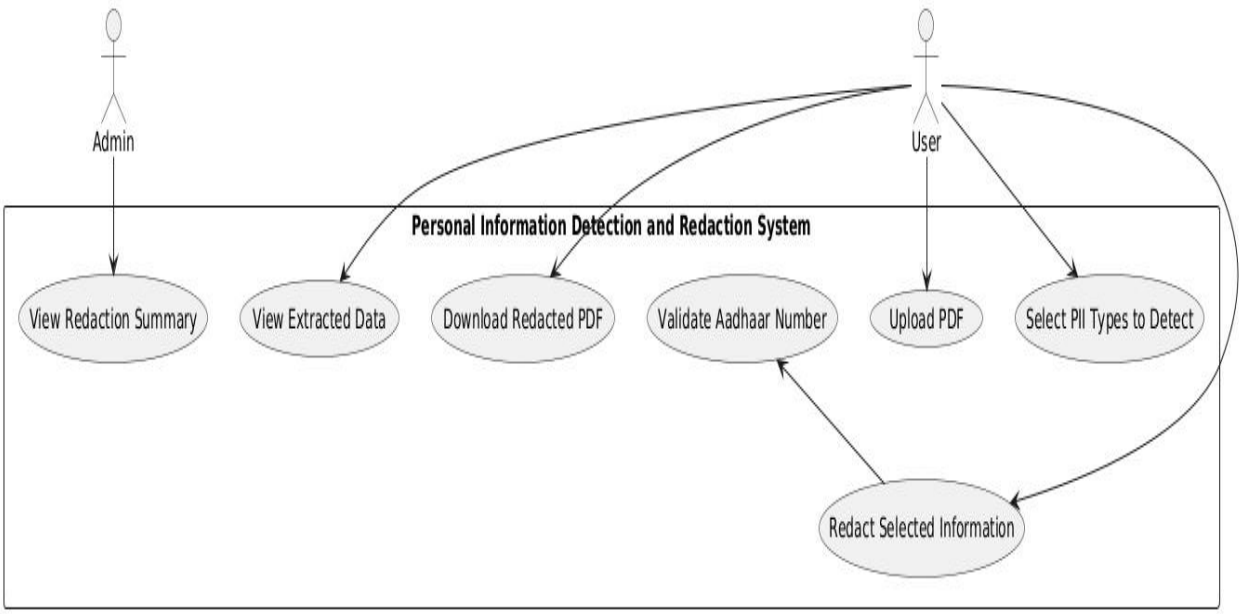


Figure-4.3.2.1 Use Case Diagram

### 4.3.3 COMPONENT DIAGRAM:

A component diagram, also known as a UML component diagram, describes the organization and wiring of the physical components in a system. Component diagrams areoften drawn to help model implementation details and double-check that every aspect of the system's required functions is covered by planned development.

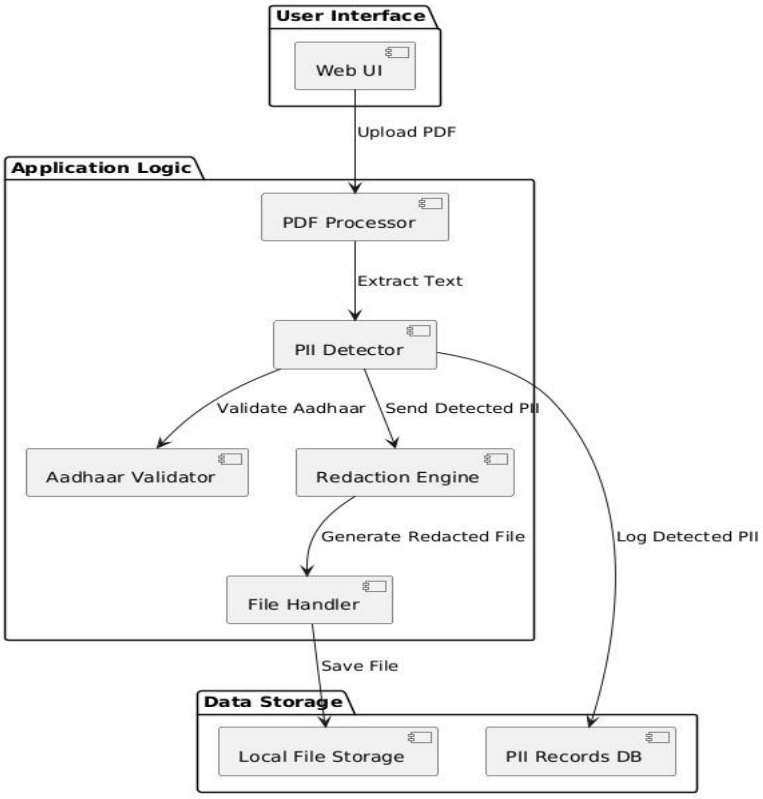


Figure-4.3.3.1 Component Diagram

### 4.3.4 SEQUENCE DIAGRAM:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, eventscenarios, and timing diagrams.

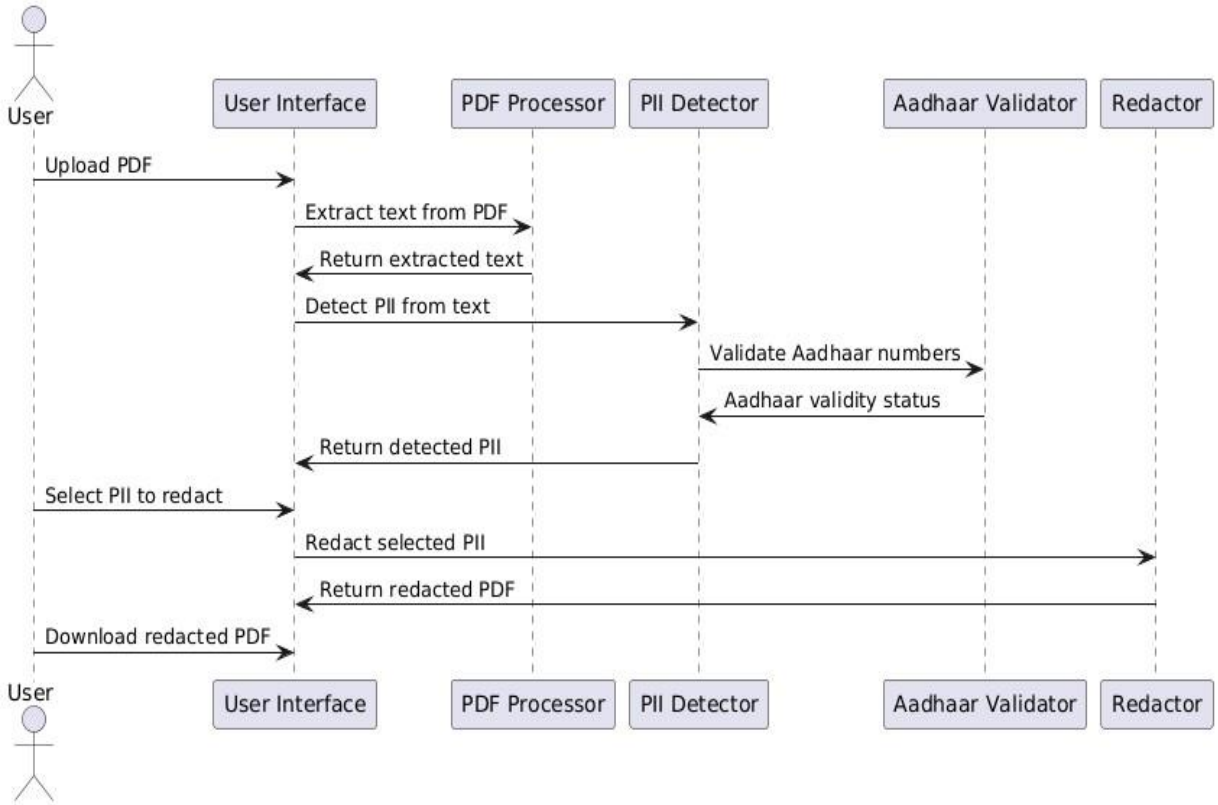


Figure-4.3.4.1 Sequence Diagram

### 4.3.5 ACTIVITY DIAGRAM:

In UML, an activity diagram is used to display the sequence of activities. Activity diagrams show the workflow from a start point to the finish point detailing the many decision paths that exist in the progression of events contained in the activity.

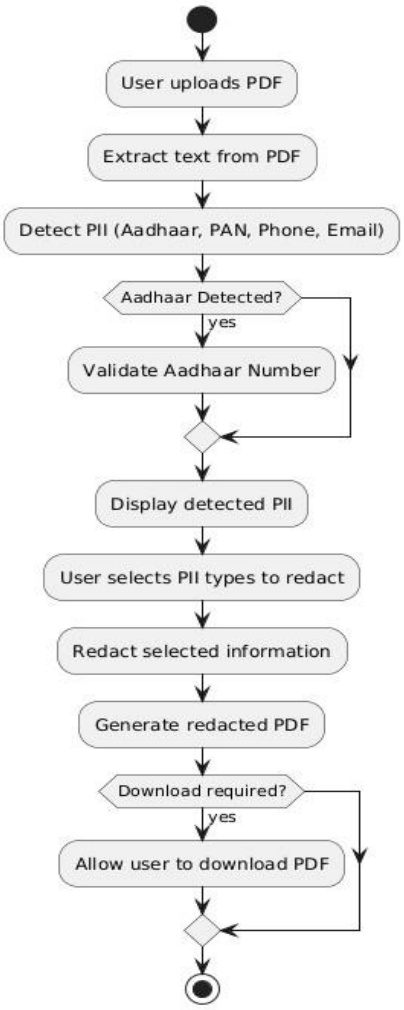


Figure-4.3.5.1 Activity Diagram

## 4.4 TECHNOLOGY DESCRIPTION

### Flask

Flask is a lightweight and flexible Python web framework used for developing web applications. It follows the WSGI standard and provides a simple yet powerful structure for building backend logic. Flask is ideal for projects that require rapid development and customization, as it allows developers to integrate external libraries and modules easily. In this project, Flask handles the backend operations such as receiving PDF uploads, processing detection requests, managing redaction workflows, and returning the final redacted files to the user through a web interface.

### PyMuPDF (fitz)

PyMuPDF, also known as fitz, is a Python binding for the MuPDF library that allows high-performance manipulation of PDF files. It supports extracting text, images, and metadata, as well as adding annotations or redactions. In this project, PyMuPDF is used to locate and irreversibly redact sensitive information while preserving the original document's layout and readability. Its precise coordinate-based operations ensure that redacted data is completely obscured and cannot be recovered.

### Regular Expressions (Regex)

Regular expressions provide a powerful way to search for patterns within text. In this project, regex is used to detect specific formats of Personally Identifiable Information (PII), such as Aadhaar numbers, PAN numbers, phone numbers, and email addresses. By combining pattern matching with custom validation rules—particularly for Aadhaar—the system reduces false positives and ensures higher detection accuracy.

### Custom Validation Logic

In addition to regex detection, custom validation algorithms are implemented to verify the authenticity of detected patterns. For example, Aadhaar numbers are validated using checksum logic to ensure the detected number is structurally correct. This extra layer of validation minimizes false detections.

# CHAPTER 5

## 5.1 IMPLEMENTATION

### Development Environment Setup

The implementation process begins with setting up the development environment, which includes installing essential software and libraries such as Python and OpenCV. OpenCV is a critical library for computer vision tasks and provides functions for image and video processing. To support the development and testing of the facial recognition system, ensure that all dependencies and tools are correctly installed and configured. This setup provides the foundation for developing the face recognition and detection algorithms.

### PDF Upload and Preprocessing

When a PDF is uploaded by the user, the system temporarily stores it in a secure local directory. PyMuPDF is used to parse the document, extract textual data, and prepare it for PII detection. Preprocessing involves normalizing text content and segmenting it into analyzable blocks, ensuring that detection algorithms can efficiently process the content without performance bottlenecks.

### Automated PII Detection

The system uses optimized regular expressions to detect Aadhaar numbers, PAN numbers, phone numbers, and email addresses. To reduce false positives, a custom validation logic is implemented—such as checksum verification for Aadhaar numbers—to confirm the authenticity of detected patterns. This multi-layer detection approach increases both accuracy and reliability.

## User-Controlled Redaction

After detection, the system presents the identified PII types to the user. The user can select which categories of information should be redacted, offering flexibility instead of forcefully removing all sensitive data. This feature is implemented via an interactive HTML form linked to Flask routes, ensuring a seamless backend–frontend integration.

## Redaction Process

Once the user confirms the selection, PyMuPDF performs coordinate-based redaction directly on the PDF. This process ensures that the redacted data is completely removed rather than just hidden. The output PDF maintains the original document layout and is stored temporarily until download.

## Security and File Management

The application is designed with offline capability and does not transmit or log user data. Temporary files are automatically deleted after the download or upon session timeout. This ensures user privacy while keeping storage usage minimal.

## Integration and Testing

The system undergoes testing across different PDF formats and text layouts to validate detection accuracy and redaction reliability. Multiple test cases, including documents with mixed-language text and varying font styles, are used to ensure the regex patterns and validation logic function correctly.

## Deployment

The application can be deployed locally without an internet connection. Users simply run the Flask app and access it via a web browser at http://localhost:5000. Since no external APIs are required, the deployment process is straightforward and highly secure.

### Evaluation and Fine-Tuning

Post-deployment, the system is evaluated based on detection precision, redaction accuracy, and processing speed. Fine-tuning is carried out by adjusting regex patterns, refining validation logic, and optimizing PyMuPDF operations. The iterative improvement process ensures that the system remains robust and adaptable to future requirements, such as the integration of OCR and AI-based entity recognition.

## 5.2 MODULES:

### File Upload Module

The File Upload Module provides a user-friendly interface for uploading PDF files containing potentially sensitive information. It validates the file format to ensure only PDFs are accepted and securely stores them temporarily for processing. This module forms the entry point for the entire redaction process.

### PDF Text Extraction Module

The PDF Text Extraction Module uses the PyMuPDF (fitz) library to read the uploaded PDF and extract its textual content. It handles both structured and unstructured text layouts, preserving positional coordinates for each detected text span. This positional data is essential for accurate redaction.

### PII Pattern Detection Module

The PII Pattern Detection Module scans extracted text using Regular Expressions (Regex) to identify predefined sensitive data formats, including:

- Aadhaar Numbers: Validated using the Verhoeff checksum algorithm for accuracy.

- PAN Numbers: Pattern-matched to standard alphanumeric structure.
- Phone Numbers: Matched against Indian mobile number formats.
- Email Addresses: Detected using robust email-matching patterns.

## File Download Module

The File Download Module allows the user to securely download the redacted PDF. The file is stored temporarily on the server and automatically removed after download to maintain confidentiality and avoid unnecessary storage.

## Web Application Interface Module

The Web Application Interface Module, built with Flask (backend) and HTML/CSS (frontend), provides a responsive and easy-to-use environment for interaction. It allows users to upload files, view redaction results, and download processed documents, ensuring smooth end-to-end operation.

## Testing and Validation Module

The Testing and Validation Module ensures that PII detection and redaction functions operate correctly across various PDF structures. It uses test cases containing real and mock data to verify regex accuracy, Aadhaar validation logic, and redaction placement before deployment.

## 5.3 EXECUTABLE CODE

### App.py

```
from flask import Flask, request, render_template, send_file, redirect, url_for
import fitz  # PyMuPDF for text positioning
import os
import re

app = Flask(_name_)

# Aadhaar validation
def validate_aadhaar_detection(text, match_start, match_end):
    """Validate Aadhaar detection by checking the next non-space character."""
    next_index = match_end
    while next_index < len(text) and text[next_index].isspace():
        next_index += 1
    # Ensure the next non-space character is NOT a digit
    if next_index < len(text) and text[next_index].isdigit():
        return False
    return True

# PII Detection Logic
def detect_pii_with_validation(text):
    """Detect PII with extra Aadhaar validation."""
    patterns = {
        'Aadhaar': r'\b\d{4}\s\d{4}\s\d{4}\b',
        'PAN': r'\b[A-Z]{5}\d{4}[A-Z]\b',
        'Phone Number': r'\b\d{10}\b',
        'Email': r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'
    }

    detected_pii = {}
    for key, pattern in patterns.items():
        matches = [(m.start(), m.end(), m.group()) for m in re.finditer(pattern, text)]
        valid_matches = []
        for match_start, match_end, match_value in matches:
            if key == 'Aadhaar' and not validate_aadhaar_detection(text, match_start, match_end):
                continue  # Skip invalid Aadhaar matches
```

```
            valid_matches.append(match_value)
        if valid_matches:
            detected_pii[key] = valid_matches

    return detected_pii

# Redaction Logic
def redact_pdf(input_path, output_path, pii_to_redact):
    """Redact selected PII in the PDF and save as a new file."""
    pdf_document = fitz.open(input_path)

    for page_number in range(len(pdf_document)):
        page = pdf_document[page_number]
        text = page.get_text()

        # Detect PII on the page
        detected_pii = detect_pii_with_validation(text)
        if detected_pii:
            for pii_type, values in detected_pii.items():
                # Only redact selected PII
                if pii_type not in pii_to_redact:
                    continue
                for value in values:
                    areas = page.search_for(value)  # Locate the text position
                    for area in areas:
                        # Draw a black rectangle over the PII
                        page.add_redact_annot(area, fill=(0, 0, 0))
                        page.apply_redactions()

    # Save the redacted PDF
    pdf_document.save(output_path)
    pdf_document.close()

# Flask Routes
@app.route('/')
def home():
    return render_template('upload.html')

@app.route('/upload', methods=['POST'])
def upload_file():
```

```
    if 'file' not in request.files:
        return "No file uploaded", 400

    file = request.files['file']
    if file.filename == '':
        return "No file selected", 400

    input_path = os.path.join('uploads', file.filename)
    os.makedirs('uploads', exist_ok=True)
    file.save(input_path)

    # Extract text and detect PII
    text = ""
    pdf_document = fitz.open(input_path)
    for page in pdf_document:
        text += page.get_text()
    pdf_document.close()

    detected_pii = detect_pii_with_validation(text)

    return render_template('select_pii.html', pii_types=list(detected_pii.keys()), input_file=file.filename)

@app.route('/redact', methods=['POST'])
def redact_file():
    pii_to_redact = request.form.getlist('pii_types')  # Get selected PII types
    input_file = request.form['input_file']
    input_path = os.path.join('uploads', input_file)
    output_path = os.path.join('uploads', 'redacted_' + input_file)

    # Redact the selected PII types
    redact_pdf(input_path, output_path, pii_to_redact)

    return send_file(output_path, as_attachment=True)

if _name_ == '_main_':
    app.run(debug=True)
```

## Select_pii.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Select PII to Redact</title>
    <style>
        body {
            font-family: 'Arial', sans-serif;
            background-color: #f9f9f9;
            color: #333;
            margin: 0;
            padding: 0;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
        }

        .container {
            text-align: center;
            background-color: #fff;
            padding: 30px 40px;
            border-radius: 10px;
            box-shadow: 0 4px 10px rgba(0, 0, 0, 0.1);
            width: 90%;
            max-width: 500px;
        }

        h1 {
            font-size: 24px;
            margin-bottom: 20px;
        }

        form {
            display: flex;
            flex-direction: column;
```

```
        align-items: flex-start;
    }

    label {
        margin: 10px 0;
        font-size: 16px;
    }

    input[type="submit"] {
        align-self: center;
        margin-top: 20px;
        background-color: #0078D7;
        color: #fff;
        padding: 10px 20px;
        font-size: 16px;
        border: none;
        border-radius: 5px;
        cursor: pointer;
    }

    input[type="submit"]:hover {
        background-color: #005BBB;
    }
  </style>
</head>
<body>
  <div class="container">
    <h1>The following is the detected sensitivie information</h1>
    <h1>Select PII to Redact</h1>
    <form action="/redact" method="POST">
        <input type="hidden" name="input_file" value="{{ input_file }}">
        {% for pii_type in pii_types %}
        <label>
            <input type="checkbox" name="pii_types" value="{{ pii_type }}"> {{ pii_type }}
        </label>
        {% endfor %}
        <input type="submit" value="Redact Selected PII">
    </form>
  </div>
</body>
```

```
</html>
```

## Upload.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>PII Redaction Tool</title>
    <style>
        body {
            font-family: 'Arial', sans-serif;
            background-color: #f9f9f9;
            color: #333;
            margin: 0;
            padding: 0;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
        }

        .container {
            text-align: center;
            background-color: #fff;
            padding: 30px 40px;
            border-radius: 10px;
            box-shadow: 0 4px 10px rgba(0, 0, 0, 0.1);
            width: 90%;
            max-width: 500px;
        }

        h1 {
            font-size: 24px;
            margin-bottom: 20px;
        }

        p {
```

```
            font-size: 16px;
            margin-bottom: 30px;
        }

        form {
            display: flex;
            flex-direction: column;
            align-items: center;
        }

        input[type="file"] {
            margin-bottom: 20px;
            padding: 10px;
            font-size: 14px;
            border: 1px solid #ccc;
            border-radius: 5px;
            width: 100%;
        }

        input[type="submit"] {
            background-color: #0078D7;
            color: #fff;
            padding: 10px 20px;
            font-size: 16px;
            border: none;
            border-radius: 5px;
            cursor: pointer;
        }

        input[type="submit"]:hover {
            background-color: #005BBB;
        }
    </style>
</head>
<body>
    <div class="container">
        <h1>PII Redaction Tool</h1>
        <p>Upload a PDF file to redact sensitive information.</p>
        <form action="/upload" method="POST" enctype="multipart/form-data">
```

```
 <input type="file" name="file" accept=".pdf" required>
          <input type="submit" value="Upload">
       </form>
    </div>
</body>
</html>
```

# CHAPTER - 6

# TESTING

## 6.1 TESTING DEFINITION:

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

## 6.2 Unit Testing

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

**Test strategy and approach**

Field testing will be performed manually and functional tests will be written in detail.

**Test objectives**

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

**Features to be tested**

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

## Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

**Test Results:** All the test cases mentioned above passed successfully. No defects encountered.

## Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

**Test Results:** All the test cases mentioned above passed successfully. No defects encountered.

Since, the grey box testing includes access to internal coding for designing test cases. Grey box testing is performed by a person who knows coding as well as testing.

## Outcomes Possible:

**Pass:** The test case successfully validates the expected behavior of the application, indicating that specific functionality works as intended.

**Fail:** The test case fails to validate the expected behavior, indicating a defect or issue in the application. This outcome requires further investigation and fixing the identified problem.

**Error:** An error occurs during the test execution due to unexpected system behavior or exceptions. This could indicate a bug or potential issue that needs to be addressed.

**Blocked:** The test case cannot be executed due to external dependencies or environmental constraints. This outcome indicates that the test case is blocked and cannot be validated at the moment.

**Skipped:** The test case is intentionally skipped from execution, typically due to low priority or specific conditions that prevent it from being applicable at the current stage of testing.

# CHAPTER - 7
# RESULTS

- After running the program we can see the following results
- **Home Page**

  The Home Page serves as the entry point to the web application. It offers a clean, user-friendly interface that guides users toward uploading and processing PDF files containing sensitive personal information. As shown in Figure 7.1, the design emphasizes simplicity and accessibility to ensure smooth user interaction.

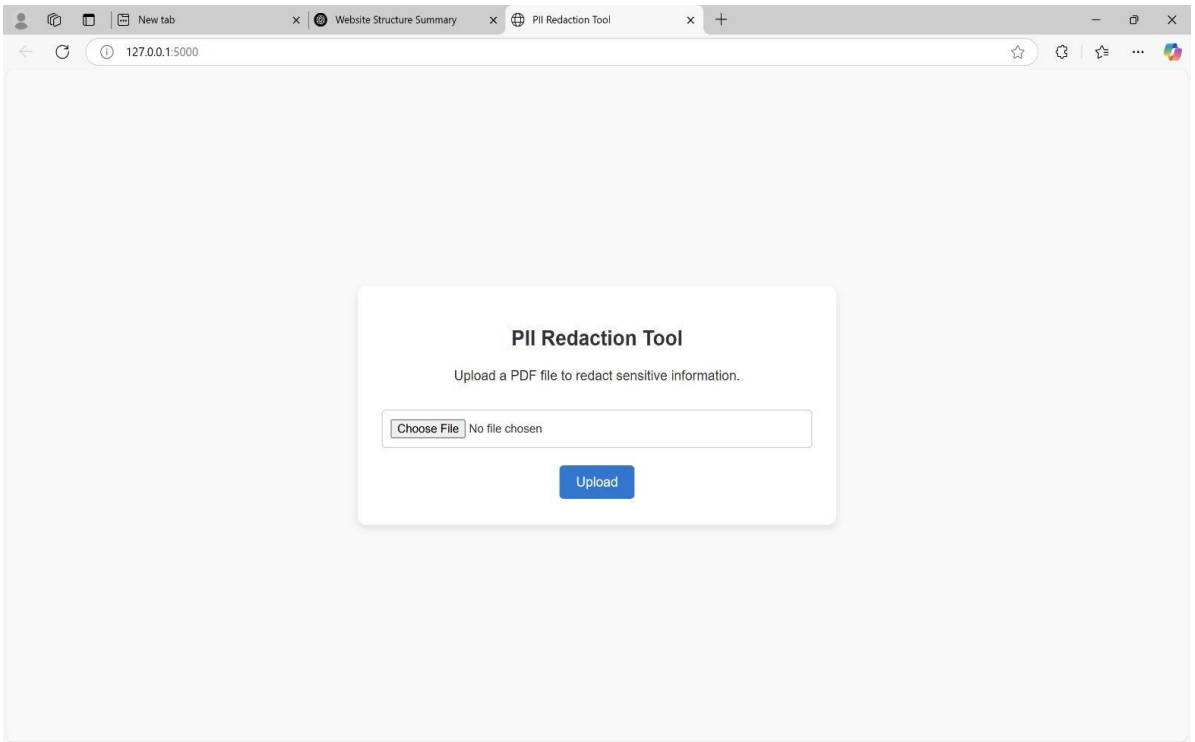


Figure-7.1 Home Page Interface

- **Upload PDF Page**

This page enables users to upload their PDF documents securely. It includes input validation and feedback messages to confirm successful uploads or alert users about errors. The interface is intuitive,allowing even non-technical users to proceed with ease.

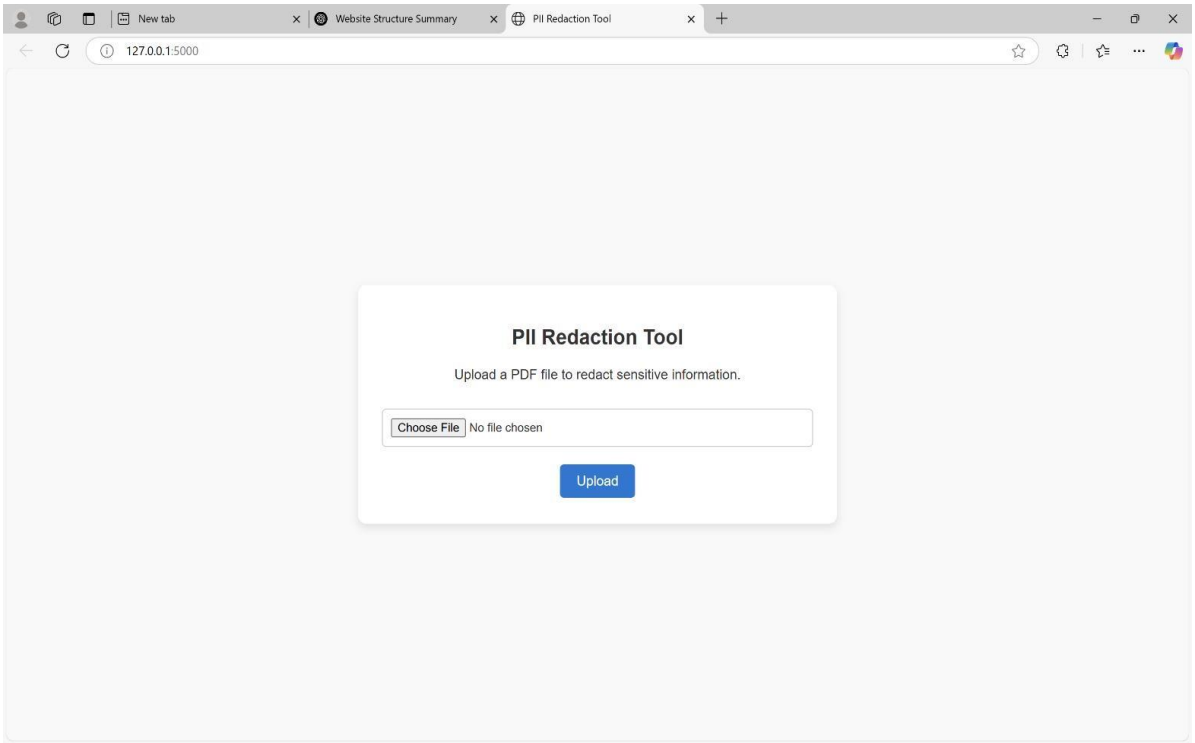


Figure-7.2 PDF upload Section

- **Detect Personal Information**

After uploading, this page displays the detected sensitive information including Aadhaar numbers, PAN numbers, phone numbers, and email IDs. It allows users to review the findings before proceeding to redaction.

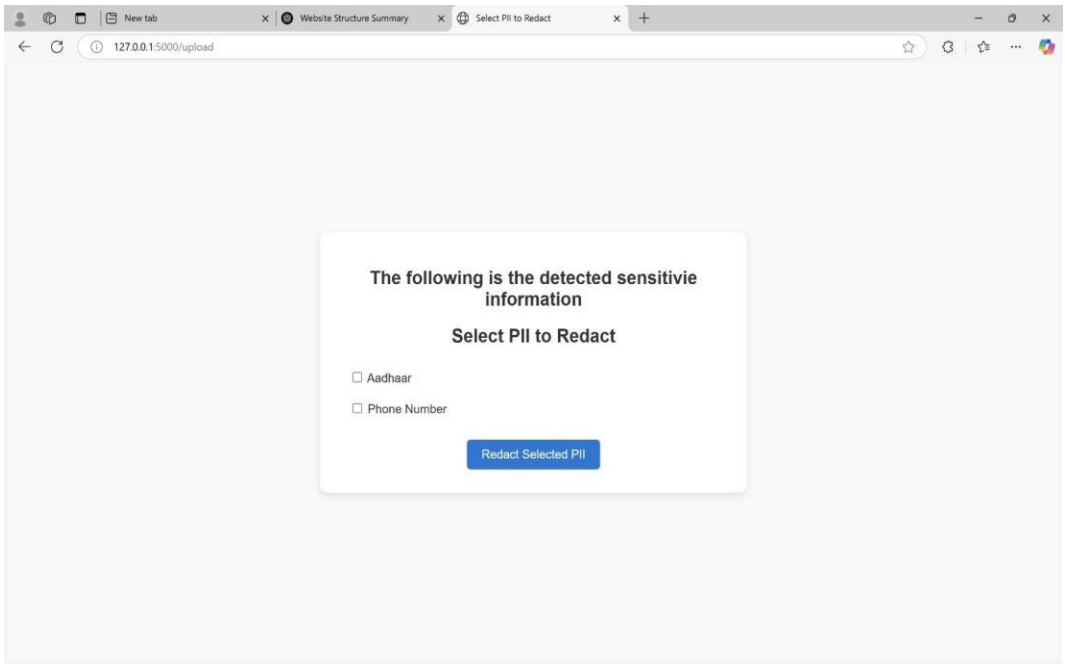


Figure-7.3 Detected Information Display

- **Download Redacted PDF**

Once the user confirms the redaction, this feature allows them to download the final redacted PDF file. The redacted content is irreversibly removed to maintain privacy.

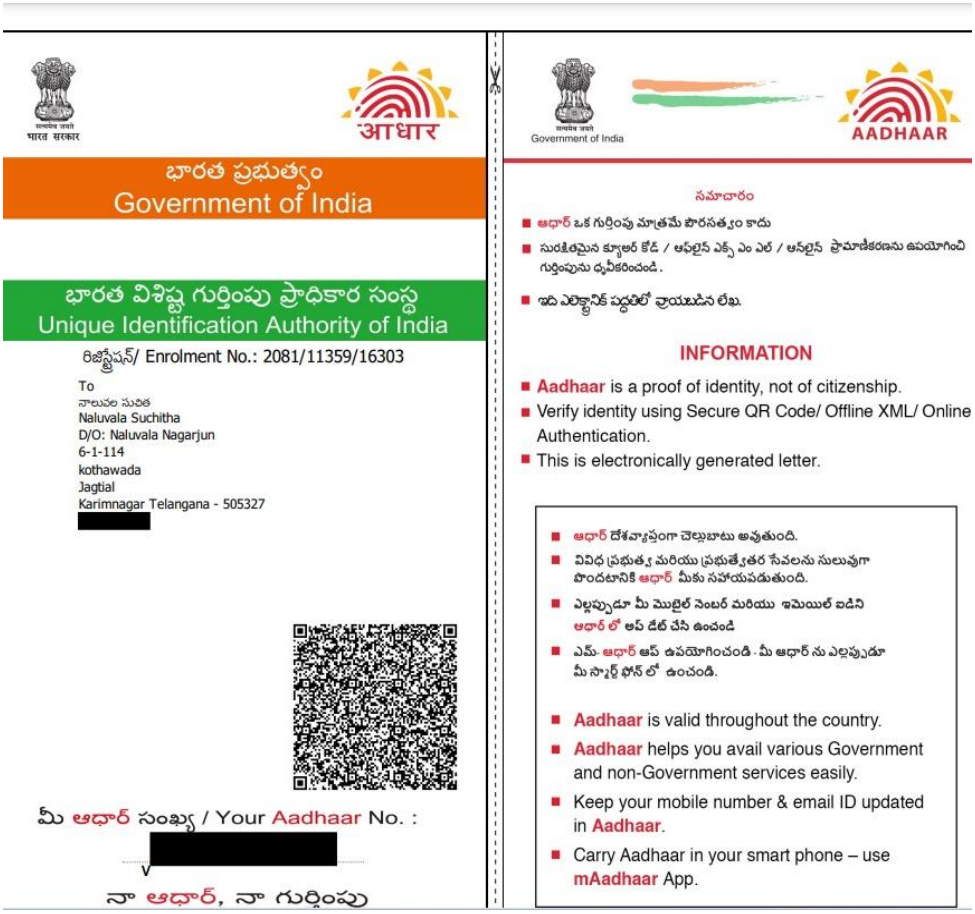


Figure-7.4 Download Redacted PDF

# CHAPTER 8

## 8.1 CONCLUSION

In an era where digital privacy is paramount, the Personal Information Detection and Redaction System emerges as a practical, secure, and efficient solution for safeguarding sensitive data in documents. Designed with a strong focus on privacy, this system empowers users to automatically detect and selectively redact personal identifiers—such as Aadhaar numbers, PAN numbers, phone numbers, and email addresses—from PDF files with a high degree of accuracy.This project not only reflects technical proficiency in natural language processing and document processing but also demonstrates a responsible approach to privacyaware software development. With its modular architecture and potential for future enhancements like OCR and AI-based entity recognition, the system lays a strong foundation for scalable and intelligent redaction tools in the digital age.

## 8.2 FUTURE SCOPE

The future scope of the project encompasses multiple enhancements, including AI-based entity recognition for more precise detection of sensitive information, customizable redaction options to allow users to choose specific data types for masking, and multi-file batch processing to handle large volumes of documents efficiently. It also involves integrating user authentication with history tracking to monitor activities, implementing role-based access control for secure and hierarchical usage, and developing a cross-platform desktop application to ensure compatibility across operating systems. Optional cloud integration can be added for remote accessibility and storage, along with multi-language support to cater to users from different regions. Additionally, the project can include automated report generation for detailed output summaries and seamless OCR integration to accurately process scanned documents and images.

# REFERENCES


- Jaikishan jaikumar,mohana,pavan kumar suresh,"privacy-preserving personal identifiable information (PII) lable detection ",ICCNT IEEE ,2023
- Aarush Verma,Bhavik Bardia,Lokesh Talluri,Yashwi J Rai,Raghavendra K,"Redacting Personal Identifiable Information",International journal of novel research and development (UNRD),Volume 9, 5 May 2024,PP:1-5,ISSN:2456-4184.
- A. A. Anwar, M. N. Khan, and M. Yousuf, "A Review of Techniques to Detect and Redact Sensitive Information from Documents," IEEE Access, vol. 9, pp. 11234–11250, 2021.
- M. Rath, S. Choudhary, and A. Das, "Sensitive Data Detection using Regular Expressions and Machine Learning," in Proc. 2020 Int. Conf. on Cyber Security and Protection of Digital Services (Cyber Security), Dublin, Ireland, 2020,pp.1–6.

# PLAGIARISM TEST

Dupli Checker

Date: 09-08-2025

## Plagiarism Scan Report

17% Plagiarism

0% Exact Match

17% Partial Match

83% Unique

| | |
|---|---|
| Words | 163 |
| Characters | 1203 |
| Sentences | 6 |
| Paragraphs | 1 |
| Read Time | 1 minute(s) |
| Speak Time | 2 minute(s) |

## Content Checked For Plagiarism

In today's digital landscape, the protection of sensitive information is a growing concern, especially with the increasing exchange of documents containing personally identifiable information (PII). This project proposes a robust system for the automated detection and redaction of PII from PDF documents using a hybrid approach involving regular expressions, contextual validation, and PDF processing techniques. The system is designed to identify critical data types such as Aadhaar numbers, PAN numbers, phone numbers, ♂and email addresses, ensuring accuracy through additional validation, particularly for Aadhaar patterns. Built using flask for the web interface and PyMuPDF (fitz) for precise PDF manipulation, the system allows users to upload a document, view detected PII, choose which elements to redact, and download a clean, redacted version. Unlike conventional detection-only systems or manual redaction tools, this project offers end-to-end privacy preservation, combining accuracy, user control, and security. This solution is especially ♂useful in contexts where sensitive data must be shared without compromising ♂confidentiality, such as legal, financial, and governmental documentation.

## Matched Source